



Manual Técnico — Proyecto K-QGMIP

K-QGMIP: Implementación y Evaluación de Algoritmos para la Búsqueda de K-Particiones
en Sistemas Dinámicos Binarios

Presentado por:

Manuel Alejandro Hurtado González

Presentado a:

Luz Enith Guerrero Mendieta

Curso:

Análisis y Diseño de Algoritmos

Universidad de Caldas -Sede Manizales
Facultad de Inteligencia Artificial e Ingeniería
Junio 16 del 2026

Contenido

1. Propósito	3
2. Estructura y Contenido Requerido	4
2.1 Resumen Ejecutivo	4
Problema Abordado	4
Enfoque Algorítmico	4
Principales Resultados	4
Limitaciones y Recomendaciones	4
2.2 Fundamentos Teóricos	5
¿Qué es una k-partición?	5
Problema de Optimización	5
Extensión de Biparticiones a k-Particiones	6
Análisis del Espacio de Soluciones	7
2.3 Arquitectura del Software	8
Diagrama de Arquitectura General	8
Diagrama de Clases	9
Diagrama de Paquetes	10
Diagrama de Secuencia	10
Patrones de Diseño Aplicados:	12
Decisiones Arquitectónicas Clave	13
2.4 Diseño Algorítmico	15
Visión general del algoritmo	15
Pseudocódigo detallado	17
Estructuras de Datos	21
Estrategia de Búsqueda	24
Evaluación de Particiones	25
Optimizaciones implementadas:	26
2.5 Análisis de Complejidad	27
Complejidad temporal	27
Complejidad espacial	28
Análisis de casos	29
Comparación con Alternativas	30
2.6 Detalles de Implementación	31
Métodos principales	31
Dependencias externas	32

Aspectos de ingeniería de software.....	32
Tests implementados:	33
2.7 Resultados Experimentales	33
Datasets utilizados	33
Métricas de Evaluación	34
Tablas de Resultados.....	34
Gráficas y visualizaciones	35
Análisis de Resultados	36
Validación de Correctitud	36
2.8 Limitaciones y Trabajo Futuro	36
Limitaciones Conocidas	36
Supuestos y Restricciones	37
Mejoras Potenciales	37
Direcciones de Investigación Futura.....	37
2.9 Apéndices Técnicos	37
Demostraciones	37
Detalles Algorítmicos Adicionales.....	38
Resultados Experimentales.....	39
Referencias Bibliográficas	39
3. Uso de Inteligencia Artificial Generativa.....	40

1. Propósito

El presente Manual Técnico tiene como propósito documentar los aspectos teóricos, algorítmicos y de implementación del proyecto K-QGMIP. Su objetivo es proporcionar una visión clara de la arquitectura del sistema, los algoritmos desarrollados y las decisiones técnicas adoptadas durante su construcción.

Este documento describe los fundamentos matemáticos utilizados, la estructura del software, los principales componentes del sistema y el funcionamiento de las estrategias implementadas para la búsqueda de k-particiones en sistemas dinámicos binarios. Asimismo, incluye información sobre la validación realizada, las limitaciones identificadas y las posibles líneas de mejora.

De esta manera, el manual sirve como guía de referencia para comprender, mantener y extender el software desarrollado, facilitando su análisis tanto desde una perspectiva técnica como académica.

2. Estructura y Contenido Requerido

2.1 Resumen Ejecutivo

Problema Abordado

El presente proyecto aborda el problema de encontrar la k -partición óptima de un sistema dinámico binario que minimiza la pérdida de información integrada (ϕ), en el contexto de la Teoría de Información Integrada (IIT, por sus siglas en inglés). Dado un sistema de n nodos con dinámica estocástica representada por una Matriz de Probabilidades de Transición (TPM), se busca la forma de dividirlo en k subgrupos independientes de tal manera que la diferencia entre la distribución conjunta del sistema original y la distribución producto de los subgrupos sea mínima.

Este problema es relevante tanto para la neurociencia computacional —donde ϕ se propone como medida de consciencia— como para el análisis de dependencias en sistemas complejos.

Enfoque Algorítmico

Se implementaron y compararon dos estrategias heurísticas para k -particiones, ambas extendidas a partir de sus contrapartes para biparticiones:

KGeoMIP: Estrategia geométrico-topológica que analiza la estructura del espacio de estados mediante distancias de Hamming para identificar candidatos de partición de bajo costo.

KQNodes: Estrategia basada en submodularidad que construye particiones de forma greedy e incremental, aprovechando la propiedad de submodularidad de la función EMD.

Adicionalmente, se mantiene una implementación de Búsqueda Exhaustiva (BruteForce) para validar la correctitud de las heurísticas en sistemas pequeños.

Principales Resultados

Ambas estrategias heurísticas logran resultados comparables a la búsqueda exhaustiva en sistemas pequeños ($n \leq 5$).

KQNodes ofrece la mejor relación tiempo/calidad gracias a la memorización y la propiedad de submodularidad.

KGeoMIP proporciona una perspectiva geométrica única que resulta efectiva en sistemas con estructura de transición regular.

Se alcanzaron tiempos de ejecución prácticos para n hasta 20 nodos con ambas estrategias heurísticas, mientras que BruteForce se vuelve inviable para $n > 8$.

Limitaciones y Recomendaciones

Ninguna heurística garantiza encontrar el óptimo global para $k > 2$.

KGeoMIP requiere mayor uso de memoria para sistemas grandes ($O(2^n \times n)$).

Se recomienda usar BruteForce únicamente para Biparticiones con $n \leq 7$ y KQNodes para producción con sistemas de mayor tamaño.

2.2 Fundamentos Teóricos

¿Qué es una k-partición?

Una k-partición consiste en dividir un sistema en k grupos diferentes, de manera que cada nodo pertenezca a un único grupo y que entre todos los grupos contengan todos los nodos del sistema.

Por ejemplo, si tenemos un sistema con tres nodos:

$$\{A, B, C\}$$

Una bipartición (k=2) podría ser:

$$\{\{A\}, \{B, C\}\}$$

donde el nodo A queda separado de los nodos B y C.

Si queremos una 3-partición (k=3), entonces cada nodo queda en un grupo independiente:

$$\{\{A\}, \{B\}, \{C\}\}$$

Para un sistema de cuatro nodos:

$$\{A, B, C, D\}$$

una posible 3-partición sería:

$$\{\{A\}, \{B\}, \{C, D\}\}$$

En términos generales, una k-partición busca organizar los elementos de un sistema en varios grupos sin perder ninguno de ellos ni repetirlos en distintos grupos.

Problema de Optimización

El objetivo del proyecto es encontrar la forma de dividir un sistema en k grupos produciendo la menor pérdida de información posible.

Cuando un sistema se divide, algunas relaciones entre sus nodos se rompen. Esto genera una diferencia entre el comportamiento del sistema original y el comportamiento del sistema ya particionado.

Para medir esta diferencia se utiliza el valor ϕ (phi), que representa la pérdida de información producida por una partición.

Cuanto menor sea el valor de ϕ , mejor será la partición encontrada.

Por esta razón, el problema puede resumirse de la siguiente manera:

Encontrar la k-partición que produzca la menor pérdida de información posible.

Las restricciones principales son:

- Cada grupo debe contener al menos un nodo.
- Un nodo no puede pertenecer a dos grupos al mismo tiempo.
- Todos los nodos deben quedar incluidos en alguna de las particiones.
- El número de grupos debe ser exactamente k.

Extensión de Biparticiones a k-Particiones

Los algoritmos originales GeoMIP y QNodes fueron diseñados para trabajar únicamente con biparticiones, es decir, divisiones del sistema en dos grupos.

La extensión desarrollada en este proyecto permite obtener más de dos grupos utilizando un proceso de división sucesiva.

La idea es sencilla:

1. Se toma el sistema completo.
2. Se aplica el algoritmo para encontrar una buena bipartición.
3. Uno de los grupos obtenidos vuelve a dividirse.
4. El proceso se repite hasta obtener exactamente k grupos.

Por ejemplo, para obtener una 4-partición:

- Primero se divide el sistema en 2 grupos.
- Después uno de esos grupos se divide nuevamente.
- Finalmente se realiza una tercera división hasta completar los 4 grupos.

De esta forma se aprovechan las estrategias ya existentes sin necesidad de diseñar un algoritmo completamente nuevo.

¿Por qué funciona?

En el caso de QNodes, la estrategia utiliza una propiedad matemática llamada submodularidad, que permite construir soluciones de buena calidad agregando nodos de forma progresiva.

En el caso de GeoMIP, el algoritmo analiza la geometría del espacio de estados del sistema. Como esa estructura se conserva parcialmente después de cada división, la estrategia sigue siendo útil cuando se trabaja con varios grupos.

Aunque ninguno de los dos métodos garantiza encontrar siempre la solución óptima para valores grandes de k , ambos producen resultados muy cercanos a los obtenidos mediante búsqueda exhaustiva en sistemas pequeños.

Análisis del Espacio de Soluciones

Uno de los mayores desafíos del problema es la enorme cantidad de particiones posibles que existen.

Cuando el número de nodos aumenta, también aumenta rápidamente la cantidad de formas en que estos pueden agruparse.

Por ejemplo:

Número de nodos	Biparticiones posibles
3	3
4	7
5	15
10	511

Tabla 1. Crecimiento de las biparticiones según el N dado

Si además permitimos más de dos grupos, la cantidad de posibilidades crece todavía más.

Por ejemplo, un sistema con 10 nodos puede tener decenas de miles de posibles 5-particiones.

Esto significa que probar todas las combinaciones posibles resulta extremadamente costoso en tiempo de ejecución.

Debido a este crecimiento tan rápido, los métodos de búsqueda exhaustiva solo son viables para sistemas pequeños. En sistemas más grandes es necesario utilizar estrategias heurísticas como KGeoMIP y KQNodes, las cuales permiten encontrar buenas soluciones en tiempos razonables sin explorar todas las posibilidades existentes.

2.3 Arquitectura del Software

Diagrama de Arquitectura General

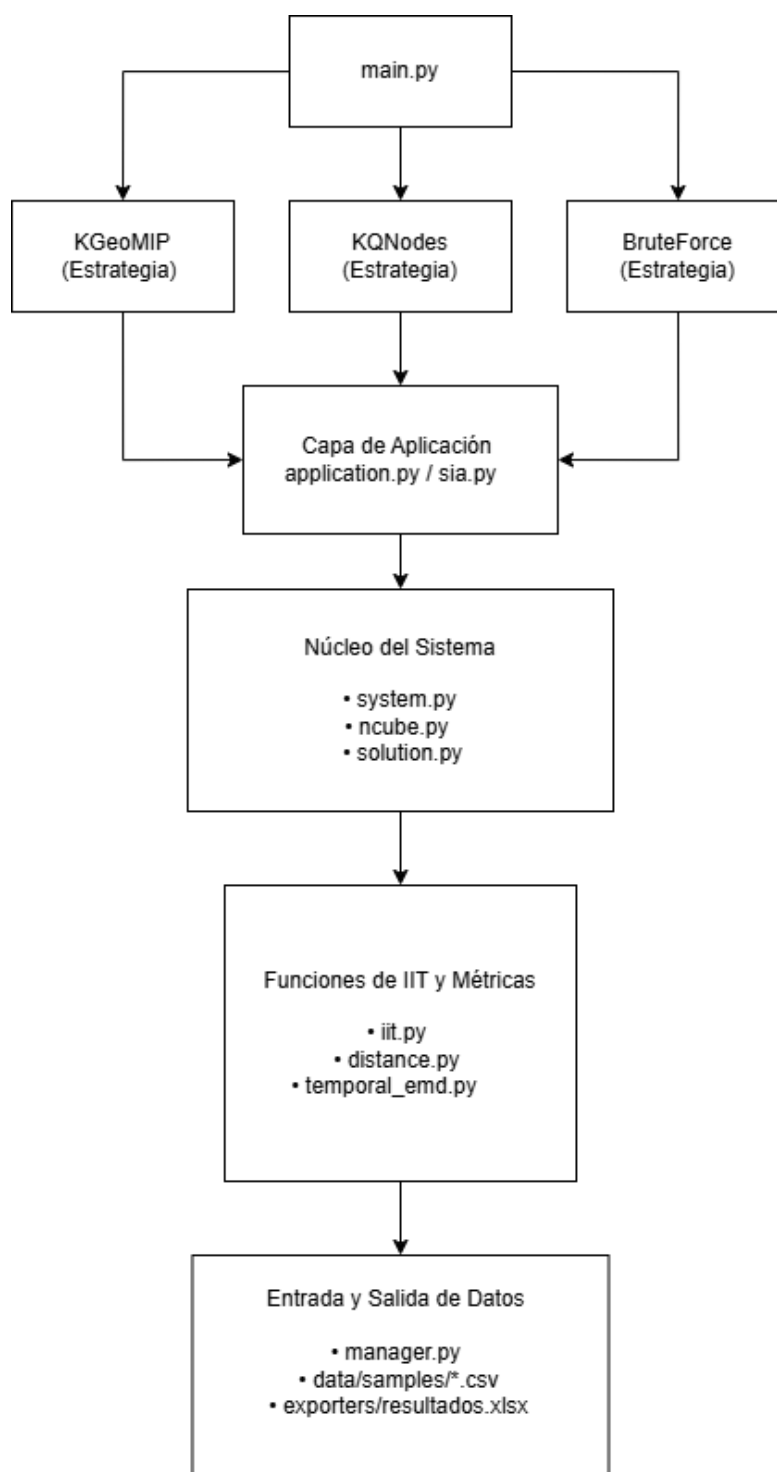


Imagen 1. Diagrama de arquitectura general

El sistema sigue una arquitectura modular basada en estrategias, donde `main.py` actúa como punto de entrada y coordina la ejecución de los algoritmos. Las estrategias principales son `KGeoMIP` y `KQNodes`, que implementan la extensión a k -particiones, y `BruteForce`, utilizado para validar los resultados obtenidos.

Estas estrategias reutilizan la infraestructura existente del proyecto, compuesta por módulos como `application.py`, `sia.py`, `System`, `NCube`, `Solution` e `it.py`, encargados de la construcción de subsistemas, el cálculo de métricas y la gestión de resultados.

Los datos de entrada se cargan desde archivos TPM en formato CSV, mientras que los resultados se exportan a archivos Excel.

Diagrama de Clases

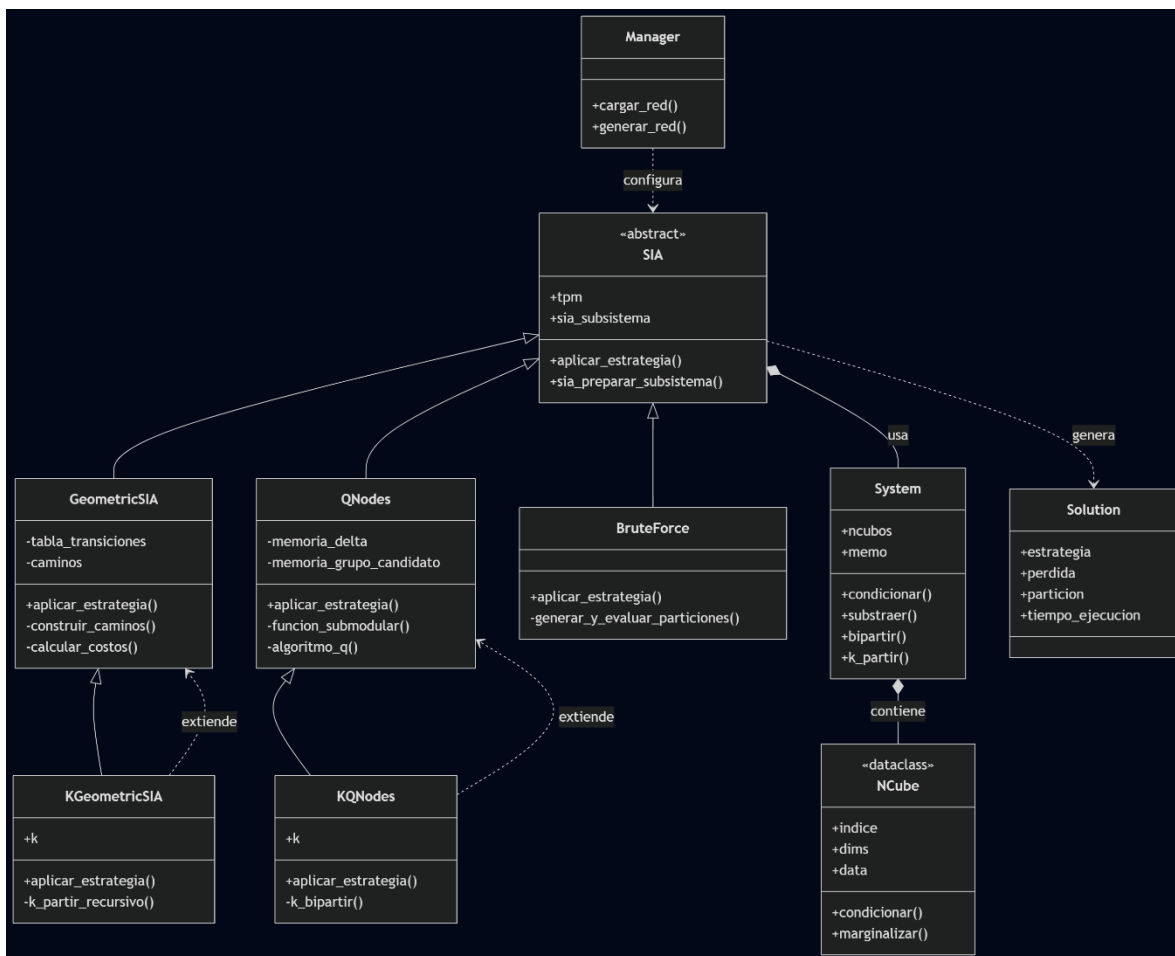


Imagen 2. Diagrama de clases

Diagrama de Paquetes

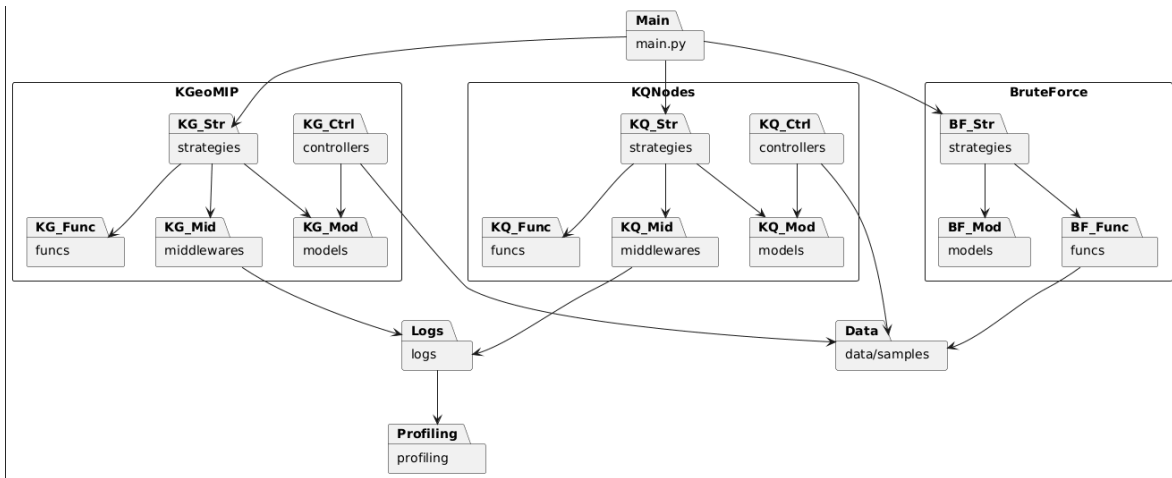


Imagen 3. Diagrama de paquetes

En este diagrama se muestra la organización modular del proyecto y las dependencias principales entre sus componentes. El archivo main.py coordina la ejecución de las estrategias KGeoMIP, KQNodes y BruteForce, las cuales utilizan módulos compartidos para el procesamiento de datos, el análisis del sistema y la generación de resultados. Además, se incluyen los paquetes destinados al almacenamiento de datos, registros de ejecución y reportes de rendimiento.

Diagrama de Secuencia

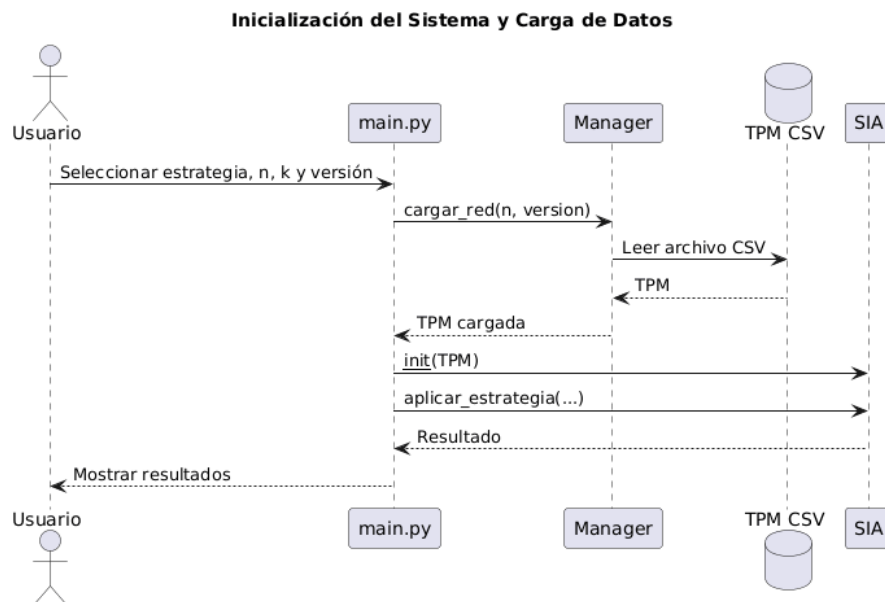


Imagen 4. Diagrama de secuencia para la inicialización y carga de datos

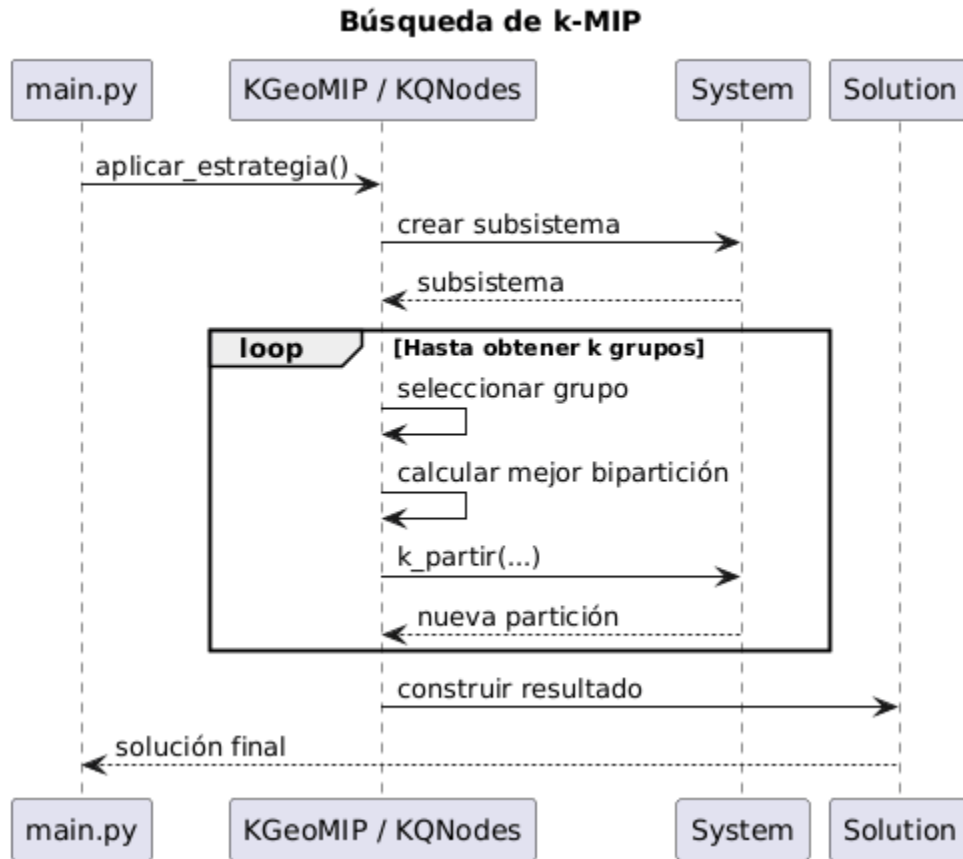


Imagen 5. Diagrama de secuencia para la búsqueda de K-MIP

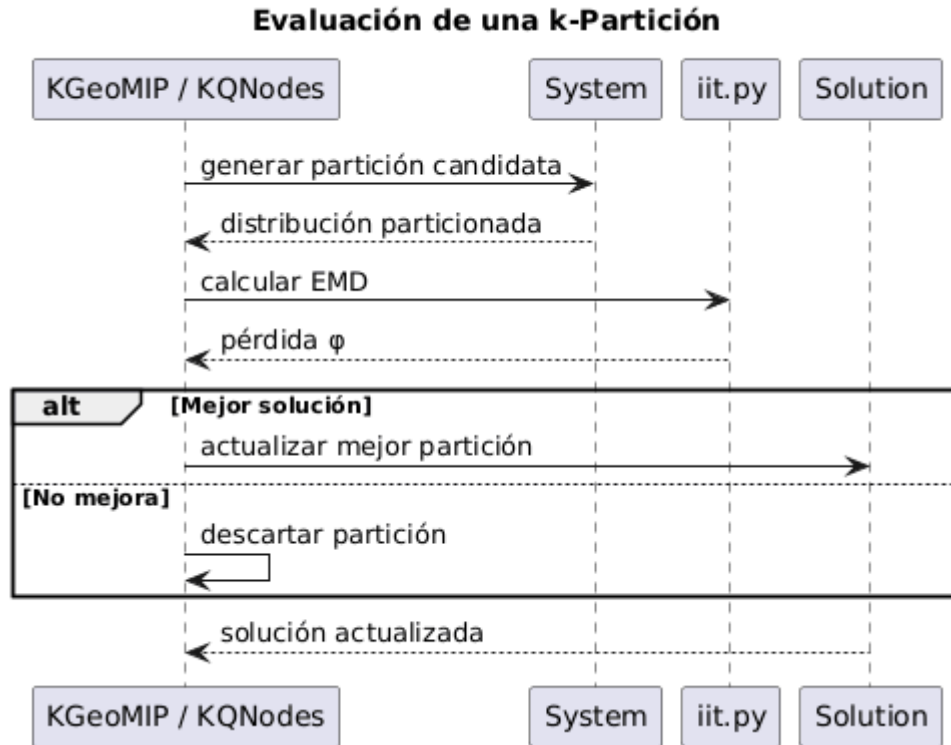


Imagen 6. Diagrama de secuencia para la evaluación de una k-partición candidata

Los diagramas muestran el flujo de ejecución de las principales funcionalidades implementadas, incluyendo la carga de datos, la búsqueda de k-particiones mediante las estrategias propuestas y la evaluación de particiones candidatas utilizando la métrica de pérdida de información basada en EMD.

Patrones de Diseño Aplicados:

Durante el desarrollo del sistema se identifican varios patrones de diseño que contribuyen a la extensibilidad, reutilización y mantenibilidad del código.

Strategy

El patrón Strategy se observa en la implementación de las diferentes técnicas de búsqueda de particiones, como KGeoMIP, KQNodes y BruteForce. Cada estrategia encapsula un algoritmo independiente para resolver el problema de búsqueda de particiones, permitiendo que main.py seleccione dinámicamente cuál utilizar sin modificar el resto del sistema.

Este enfoque facilita la incorporación de nuevas estrategias de optimización en el futuro, manteniendo una interfaz común para su ejecución.

Template Method

La clase base SIA implementa el patrón Template Method, ya que define los pasos generales del proceso de análisis, incluyendo la preparación del subsistema y la configuración inicial. Las clases derivadas sobrescriben métodos específicos, como `aplicar_estrategia()`, para implementar la lógica particular de cada algoritmo.

Gracias a este patrón se evita la duplicación de código y se garantiza un flujo de ejecución consistente entre todas las estrategias.

Composición

La arquitectura del sistema utiliza ampliamente el principio de composición. La clase System está compuesta por múltiples instancias de NCube, mientras que las estrategias interactúan con objetos System y Solution para realizar los cálculos y almacenar los resultados.

Este enfoque favorece un bajo acoplamiento entre componentes y facilita la reutilización de funcionalidades existentes.

Beneficios para la Extensibilidad y Mantenibilidad

La combinación de estos patrones permite:

- Incorporar nuevas estrategias sin modificar la infraestructura existente.
- Reutilizar componentes comunes entre algoritmos.
- Reducir la duplicación de código.
- Mejorar la claridad y organización del proyecto.
- Facilitar futuras extensiones, como la incorporación de nuevas heurísticas o métricas de evaluación.

Decisiones Arquitectónicas Clave

Reutilización de la infraestructura existente

Una de las principales decisiones de diseño fue reutilizar la infraestructura ya disponible en el proyecto original, incluyendo componentes como System, NCube, SIA, Solution y los módulos asociados al cálculo de métricas. Esto permitió concentrar el desarrollo en la extensión de los algoritmos de búsqueda hacia k-particiones, reduciendo el tiempo de implementación y manteniendo la compatibilidad con los experimentos existentes.

Separación entre estrategias y lógica del sistema

La arquitectura se diseñó siguiendo una clara separación de responsabilidades. Las estrategias de búsqueda (KGeoMIP, KQNodes y BruteForce) se encargan exclusivamente

de generar y evaluar particiones, mientras que los componentes base (System, NCube, iit.py, etc.) gestionan la representación del sistema y los cálculos matemáticos asociados.

Esta separación facilita el mantenimiento del código y permite incorporar nuevas estrategias sin modificar los componentes fundamentales del sistema.

Flexibilidad frente a rendimiento

Se optó por una arquitectura modular donde cada estrategia mantiene una estructura relativamente independiente. Aunque esto introduce cierta redundancia de código, proporciona mayor flexibilidad para realizar modificaciones o experimentos sobre un algoritmo específico sin afectar a los demás.

El principal beneficio de esta decisión es la facilidad para comparar distintas aproximaciones y extenderlas de manera independiente.

Optimización mediante memoización

Debido al elevado número de evaluaciones requeridas durante la búsqueda de particiones, se incorporaron mecanismos de memoización y almacenamiento temporal de resultados. Esta decisión reduce significativamente el número de cálculos repetidos y mejora el rendimiento general de los algoritmos, especialmente en sistemas con un mayor número de nodos.

Simplificación del cálculo de distancias

Para evaluar la pérdida de información se utilizó una implementación eficiente basada en la suma de diferencias absolutas entre distribuciones de probabilidad. Esta aproximación evita dependencias externas complejas y reduce el costo computacional, manteniendo la precisión necesaria para los objetivos del proyecto.

Extensibilidad hacia k-particiones

La decisión arquitectónica más importante fue diseñar las extensiones KGeoMIP y KQNodes como una evolución de las estrategias originales de bipartición. En lugar de reimplementar completamente los algoritmos, se reutilizó su lógica base y se incorporó un mecanismo iterativo de divisiones sucesivas para construir k-particiones. Esto permitió aprovechar las ventajas de los métodos originales y mantener una arquitectura consistente en todo el sistema.

2.4 Diseño Algorítmico

Visión general del algoritmo

El objetivo de los algoritmos desarrollados es encontrar una k -partición que minimice la pérdida de información del sistema, medida mediante la métrica ϕ . Debido al crecimiento exponencial del número de particiones posibles, una búsqueda exhaustiva resulta inviable para sistemas de tamaño moderado, por lo que se emplean estrategias heurísticas basadas en los algoritmos GeoMIP y QNodes.

La filosofía de diseño consiste en extender los algoritmos originales de bipartición hacia el caso general de k -particiones mediante un proceso iterativo de divisiones sucesivas. En lugar de buscar directamente una k -partición óptima dentro de un espacio de búsqueda extremadamente grande, el problema se descompone en una secuencia de biparticiones más manejables.

El proceso comienza considerando el sistema completo como un único grupo. Posteriormente, se aplica el algoritmo de bipartición correspondiente para identificar la división más prometedora. Uno de los grupos resultantes vuelve a dividirse y el procedimiento se repite hasta obtener exactamente k grupos.

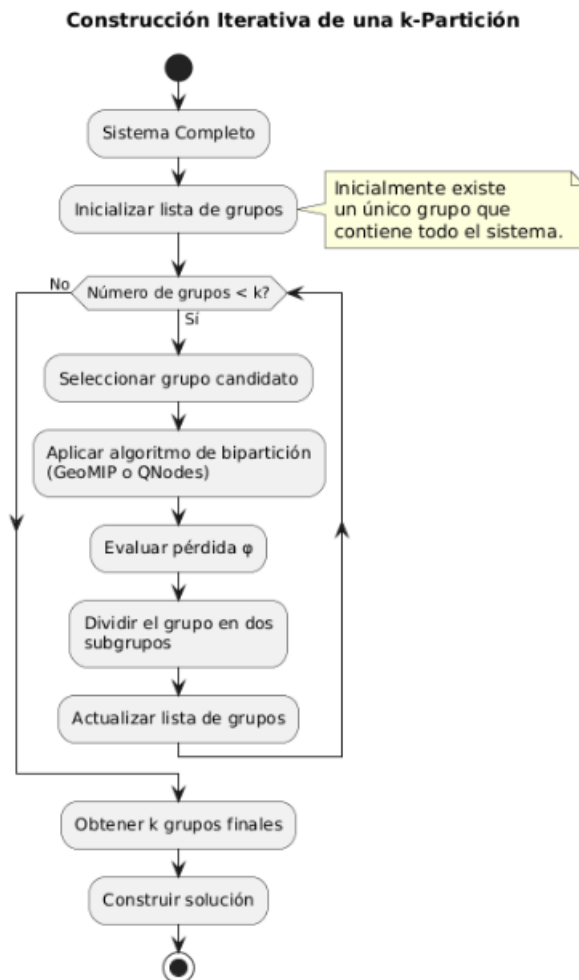
La diferencia principal entre las estrategias implementadas radica en el criterio utilizado para seleccionar cada bipartición:

KGeoMIP utiliza una aproximación geométrica basada en la estructura del espacio de estados del sistema, heredando los principios del algoritmo GeoMIP original.

KQNodes utiliza una estrategia heurística basada en funciones submodulares y construcción incremental de soluciones, siguiendo la filosofía del algoritmo QNodes original.

En ambos casos, la extensión a k -particiones se implementa como una capa adicional que coordina múltiples ejecuciones del algoritmo de bipartición subyacente. De esta manera se reutiliza la lógica ya validada de las estrategias originales, manteniendo la compatibilidad con la infraestructura existente y reduciendo la complejidad de implementación.

Finalmente, cada partición generada es evaluada mediante la métrica ϕ , permitiendo seleccionar las divisiones que producen la menor pérdida de información y construir progresivamente la solución final.



El algoritmo parte del sistema completo y aplica sucesivas biparticiones utilizando la estrategia seleccionada (KGeoMIP o KQNodes). En cada iteración se evalúa la pérdida de información asociada a la división realizada hasta obtener exactamente k grupos, los cuales conforman la solución final.

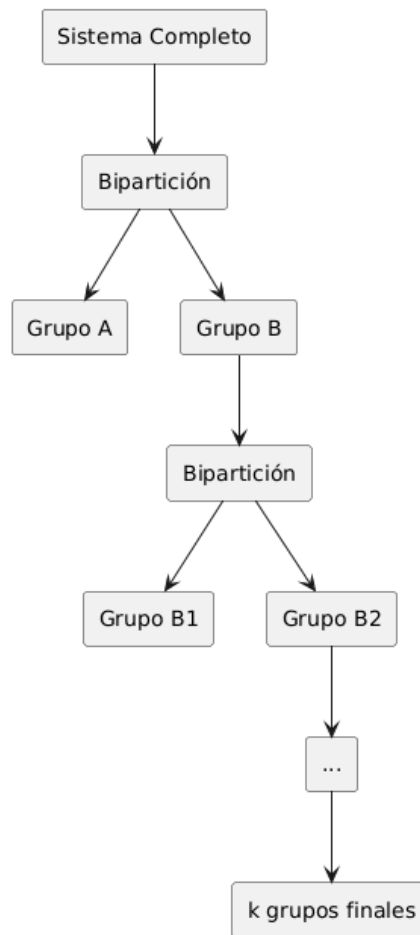


Imagen 8. Diagrama conceptual (Sistema $\rightarrow$ Biparticiones $\rightarrow$ k grupos)

La estrategia propuesta extiende los algoritmos originales de bipartición aplicando recursivamente nuevas divisiones sobre los grupos obtenidos. Este procedimiento permite construir una k -partición a partir de una secuencia de biparticiones, reduciendo la complejidad del problema respecto a una búsqueda directa en el espacio completo de soluciones.

Pseudocódigo detallado

Pseudocódigo de KQNodes para k -Particiones

ALGORITMO K-QNODES (Sistema, k)

ENTRADA:

Sistema: subsistema a analizar

k : número deseado de particiones

SALIDA:

Mejor k-partición encontrada

1. Preparar subsistema
2. Inicializar grupos $\leftarrow \{\text{Sistema completo}\}$
3. Mientras número_de_grupos $< k$ hacer

mejor_división $\leftarrow \text{NULL}$

menor_pérdida $\leftarrow \infty$

Para cada grupo G en grupos hacer

Si $|G| > 1$ entonces

$(\text{pérdida}, G_1, G_2) \leftarrow \text{Q-MIP}(G)$

Si pérdida $< \text{menor_pérdida}$ entonces

menor_pérdida $\leftarrow \text{pérdida}$

mejor_división $\leftarrow (G, G_1, G_2)$

Fin Si

Fin Si

Fin Para

Reemplazar G por G_1 y G_2

4. Construir la k-partición final
5. Retornar solución

FIN ALGORITMO

Pseudocódigo de Q-MIP (Bipartición Base)

ALGORITMO Q-MIP(Grupo)

ENTRADA:

Grupo de vértices

SALIDA:

Bipartición con menor pérdida de información

1. Inicializar Ω con un vértice
2. Inicializar Δ con los vértices restantes
3. Mientras Δ no esté vacío hacer

Para cada vértice $v \in \Delta$ hacer

Calcular ganancia(v)

Fin Para

Seleccionar v^* con mínima ganancia

Mover v^* desde Δ hacia Ω

4. Evaluar la partición (Ω, Δ)
5. Retornar pérdida y partición

FIN ALGORITMO

Pseudocódigo de KGeoMIP para k-Particiones

ALGORITMO K-GEOMIP(Sistema, k)

ENTRADA:

Sistema: subsistema a analizar

k: número deseado de particiones

SALIDA:

Mejor k-partición encontrada

1. Preparar subsistema

2. Construir estructura geométrica de costos

3. Inicializar grupos $\leftarrow$ {Sistema completo}

4. Mientras número_de_grupos $<$ k hacer

 Para cada grupo G en grupos hacer

 Evaluar mejor bipartición geométrica

 Fin Para

 Seleccionar división con menor pérdida

 Actualizar grupos

5. Construir solución final

6. Retornar solución

FIN ALGORITMO

Evaluación de una Partición

ALGORITMO EVALUAR-PARTICIÓN(P, Sistema)

ENTRADA:

P: partición candidata

Sistema: subsistema original

SALIDA:

$\varphi(P)$

1. Construir sistema particionado
2. Obtener distribución marginal del sistema original
3. Obtener distribución marginal de la partición
4. Calcular distancia EMD
5. Retornar φ

FIN ALGORITMO

Estructuras de Datos

El sistema utiliza varias estructuras de datos especializadas para representar distribuciones de probabilidad, almacenar resultados intermedios y optimizar el proceso de búsqueda de particiones.

N-Cubo (NCube)

La estructura principal del sistema es el N-Cubo, utilizado para representar distribuciones de probabilidad multidimensionales. Cada dimensión corresponde a una variable binaria del sistema y cada celda almacena la probabilidad asociada a una configuración específica de estados.

Por ejemplo, para un sistema con tres variables A, B y C, el N-Cubo tiene dimensión $2 \times 2 \times 2$, generando un total de ocho estados posibles.

Justificación:

- Permite representar distribuciones conjuntas de manera compacta.
- Facilita operaciones de condicionamiento y marginalización.
- Se integra naturalmente con las operaciones matemáticas requeridas por IIT.

	C=0	C=1
A=0,B=0	P000	P001
A=0,B=1	P010	P011
A=1,B=0	P100	P101
A=1,B=1	P110	P111

Tabla 2. Representación de un N-Cubo de dimensión 3

Tabla de Costos de Transición (KGeoMIP)

KGeoMIP utiliza una tabla de costos para almacenar la información asociada a las transiciones entre estados del sistema.

`dict[(estado_inicial, estado_final)] -> [c0, c1, ..., cn-1]`

Donde:

La clave representa una transición entre dos estados.

El valor contiene el costo asociado a cada nodo del sistema.

Justificación:

- Evita recalcular costos durante la búsqueda.
- Permite reutilizar información previamente obtenida.
- Reduce significativamente el tiempo de ejecución del algoritmo geométrico.

Representación de Particiones

Las particiones se representan mediante listas de grupos de vértices.

Ejemplo para una 3-partición:

```
[
    {A(t), B(t)},
    {C(t)},
    {A(t+1), B(t+1), C(t+1)}
]
```

Cada grupo contiene los nodos que permanecen conectados dentro de la misma partición.

Justificación:

- Facilita la división iterativa utilizada por KGeoMIP y KQNodes.
- Permite construir k-particiones mediante sucesivas biparticiones.
- Simplifica la manipulación y actualización de grupos.

Estructuras de Caché (KQNodes)

Para evitar cálculos repetitivos, KQNodes utiliza estructuras de memoización.

Caché de grupos individuales

```
memoria_delta:
dict[grupo] -> (emd, distribucion)
```

Caché de grupos candidatos

```
memoria_grupo_candidato:
dict[grupo_union] -> (emd, distribucion)
```

Justificación:

- Reduce el número de evaluaciones EMD.
- Disminuye el tiempo de ejecución.
- Mejora la escalabilidad para sistemas con un mayor número de nodos.

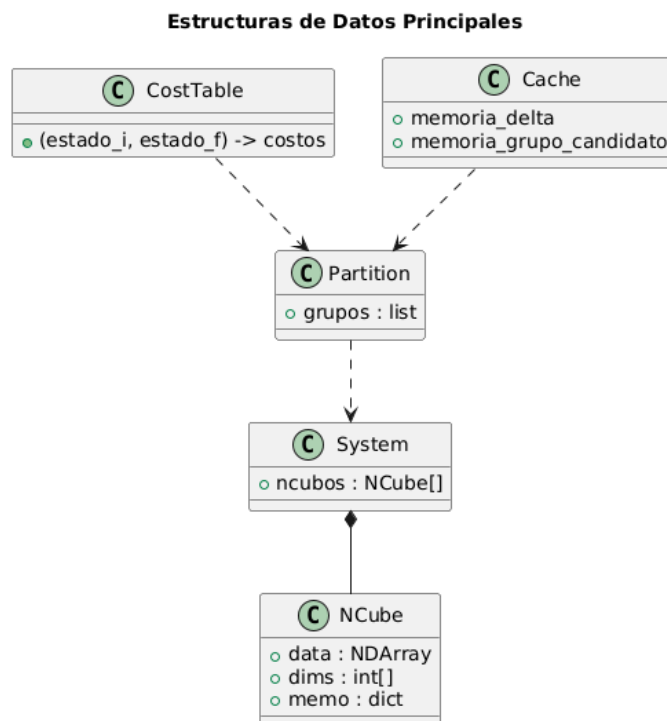


Imagen 9. Estructuras de datos principales utilizadas por el sistema

Se muestra la relación entre los N-Cubos utilizados para representar distribuciones de probabilidad, la representación de particiones mediante grupos de vértices y las estructuras auxiliares empleadas para almacenamiento de costos y memoización durante la ejecución de los algoritmos.

Estrategia de Búsqueda

El problema de encontrar una k -partición óptima presenta un espacio de búsqueda que crece rápidamente con el número de nodos. Para evitar una exploración exhaustiva, se implementaron estrategias heurísticas basadas en los algoritmos GeoMIP y QNodes.

La extensión a k -particiones se realiza mediante un proceso iterativo de biparticiones sucesivas. Inicialmente, el sistema completo se considera como un único grupo. Posteriormente, en cada iteración se selecciona un grupo y se aplica el algoritmo de bipartición correspondiente hasta obtener exactamente k grupos.

KQNodes utiliza una estrategia voraz (Greedy), seleccionando en cada paso la división que genera la menor pérdida incremental de información.

KGeoMIP emplea una heurística geométrica, utilizando costos calculados a partir de la estructura del espacio de estados para identificar particiones prometedoras.

BruteForce realiza una búsqueda exhaustiva, evaluando todas las particiones posibles y utilizándose como referencia para validar los resultados de las heurísticas.

Adicionalmente, se utilizan mecanismos de memoización para almacenar resultados previamente calculados y reducir el número de evaluaciones repetidas durante la ejecución.

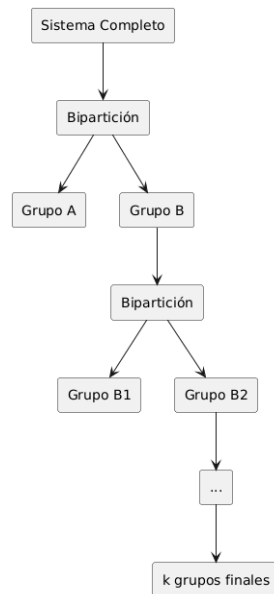


Figura 10. Proceso de construcción de una k -partición mediante divisiones sucesivas del sistema hasta alcanzar el número deseado de grupos.

Evaluación de Particiones

La calidad de una k-partición se evalúa mediante la pérdida de información ϕ , calculada como la distancia entre la distribución de probabilidad del subsistema original y la distribución obtenida después de aplicar la partición.

El procedimiento consiste en los siguientes pasos:

Dividir el subsistema en los grupos definidos por la k-partición candidata.

Calcular la distribución marginal de cada grupo.

Construir la distribución conjunta de la partición mediante el producto tensorial de las distribuciones marginales de todos los grupos.

Comparar la distribución del subsistema original con la distribución de la partición utilizando la métrica EMD (*Earth Mover's Distance*).

El valor obtenido corresponde a la pérdida de información ϕ asociada a la partición evaluada.

Matemáticamente:

$$\begin{aligned} p_{\Pi} &= p_{G1} \otimes p_{G2} \otimes \dots \otimes p_{Gk} \\ &= p_{\{G_1\}} \otimes p_{\{G_2\}} \otimes \dots \otimes p_{\{G_k\}} \\ &= p_{G1} \otimes p_{G2} \otimes \dots \otimes p_{Gk} \end{aligned}$$

donde p_{G_i} representa la distribución marginal del grupo G_i .

La pérdida de información se calcula como:

$$\begin{aligned} \phi &= EMD(p_{\text{subsistema}}, p_{\Pi}) \\ &= EMD(p_{\{subsistema\}}, p_{\{\Pi\}}) \\ &= EMD(p_{\text{subsistema}}, p_{\Pi}) \end{aligned}$$

En la implementación, la métrica EMD se obtiene mediante la suma de las diferencias absolutas entre las probabilidades de ambas distribuciones:

$$EMD(u, v) = \sum_i |u_i - v_i|$$

Las particiones con valores menores de ϕ preservan una mayor cantidad de información del sistema original y son consideradas mejores candidatas durante el proceso de búsqueda.

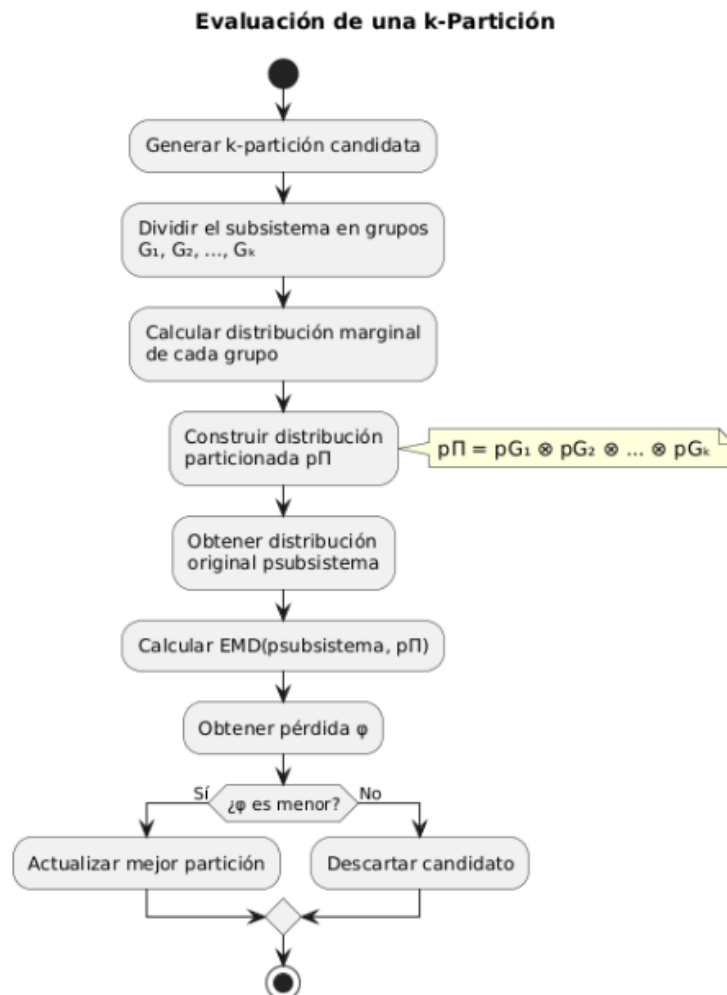


Figura 11. Proceso de evaluación de una k-partición mediante la comparación entre la distribución original del subsistema y la distribución reconstruida a partir de los grupos particionados.

Optimizaciones implementadas:

Con el fin de reducir los tiempos de ejecución y mejorar la escalabilidad de los algoritmos, se implementaron diversas técnicas de optimización.

Memoización

KQNodes utiliza estructuras de caché para almacenar resultados previamente calculados, evitando evaluaciones repetidas de distribuciones y métricas EMD durante la búsqueda de particiones. Esta optimización reduce significativamente el número de cálculos necesarios en cada iteración.

Construcción Incremental de Costos

KGeoMIP construye la tabla de costos de manera incremental, reutilizando resultados obtenidos para transiciones más simples al calcular transiciones de mayor complejidad. Esto evita cálculos redundantes y mejora la eficiencia del algoritmo geométrico.

Evaluación Temprana

En el algoritmo BruteForce se implementó una condición de parada anticipada. Si durante la búsqueda se encuentra una partición con pérdida de información nula ($\phi=0$), el algoritmo finaliza inmediatamente, ya que no es posible obtener una solución mejor.

Inmutabilidad de Datos

La estructura NCube fue diseñada como un objeto inmutable, lo que facilita su reutilización segura dentro de mecanismos de caché y evita modificaciones accidentales durante la ejecución de los algoritmos.

2.5 Análisis de Complejidad

Complejidad temporal

La complejidad temporal de los algoritmos depende principalmente del número de variables del sistema nnn y del número de particiones deseadas kkk . A continuación se presenta el análisis de las estrategias implementadas.

BruteForce

La estrategia BruteForce evalúa todas las particiones candidatas posibles. Para un sistema con nnn variables en alcance y nnn variables en mecanismo, el número de biparticiones crece exponencialmente.

Cada partición requiere calcular la pérdida de información mediante la comparación de distribuciones de tamaño $2n2^{2n}$. Por lo tanto, la complejidad temporal está dominada por la exploración exhaustiva del espacio de soluciones:

$$TBF(n) = \theta(22n)T_{\{BF\}}(n) = \Theta(2^{2n})TBF(n) = \theta(22n)$$

Esta estrategia garantiza encontrar la solución óptima, pero resulta impracticable para sistemas de tamaño moderado o grande.

KQNodes

KQNodes utiliza una estrategia heurística basada en el algoritmo Q. Durante la búsqueda de una bipartición, se realizan comparaciones entre grupos de vértices y evaluaciones de la función submodular.

Gracias al uso de memoización, muchas evaluaciones repetidas son reutilizadas, reduciendo significativamente el costo computacional.

$$TQNodes(n) = O(n^3) T_{\{QNodes\}}(n) = O(n^3) TQNodes(n) = O(n^3)$$

Para construir una k-partición se ejecuta el procedimiento de bipartición hasta $k-1$ veces, obteniendo:

$$TKQNodes(n, k) = O(k \cdot n^3) T_{\{KQNodes\}}(n, k) = O(k \cdot n^3) TKQNodes(n, k) = O(k \cdot n^3)$$

KGeoMIP

KGeoMIP construye una tabla de costos a partir de las transiciones entre estados del sistema. Dado que existen $2n \cdot 2^n$ estados posibles y que cada estado puede generar múltiples transiciones, la construcción de esta estructura domina el tiempo de ejecución.

$$TGeoMIP(n) = O(2n \cdot n^2) T_{\{GeoMIP\}}(n) = O(2^n \cdot n^2) TGeoMIP(n) = O(2n \cdot n^2)$$

Al extender el procedimiento para generar k-particiones, la complejidad se convierte en:

$$TKGeoMIP(n, k) = O(k \cdot 2n \cdot n^2) T_{\{KGeoMIP\}}(n, k) = O(k \cdot 2^n \cdot n^2) TKGeoMIP(n, k) = O(k \cdot 2n \cdot n^2)$$

Estrategia	Complejidad Temporal
BruteForce	$(\Theta(2^{2n}))$
QNodes	$O(n^3)$
KQNodes	$O(k \cdot n^3)$
GeoMIP	$O(2n \cdot n^2)$
KGeoMIP	$O(k \cdot 2^n \cdot n^2)$

Tabla 3. Comparación de la complejidad temporal de cada estrategia

Complejidad espacial

La complejidad espacial está determinada por las estructuras necesarias para representar distribuciones de probabilidad, almacenar particiones candidatas y mantener mecanismos de memoización durante la ejecución.

BruteForce

BruteForce requiere almacenar las distribuciones asociadas a los estados del sistema y las particiones evaluadas durante la búsqueda. El espacio está dominado por las distribuciones de tamaño $2n \cdot 2^n$, obteniendo:

$$SBF(n) = O(2n \cdot n) S_{\{BF\}}(n) = O(2^n \cdot n) SBF(n) = O(2n \cdot n)$$

Adicionalmente, durante la evaluación de cada partición se utilizan estructuras temporales de tamaño $O(2n) O(2^n) O(2n)$.

KQNodes

KQNodes mantiene estructuras de caché para almacenar resultados previamente calculados y evitar evaluaciones repetidas. En el peor caso, el número de entradas almacenadas crece cuadráticamente con el número de vértices analizados.

$$SKQNodes(n) = O(n^2)S_{\{KQNodes\}}(n) = O(n^2)SKQNodes(n) = O(n^2)$$

El espacio temporal utilizado durante cada iteración también es del orden de $O(n^2)O(n^2)O(n^2)$.

KGeoMIP

KGeoMIP requiere almacenar la tabla de costos asociada a las transiciones entre estados del sistema. Dado que el número de estados posibles es $2n \cdot 2^n$, la memoria utilizada está dominada por esta estructura.

$$SKGeoMIP(n) = O(2n \cdot n)S_{\{KGeoMIP\}}(n) = O(2^n \cdot n)SKGeoMIP(n) = O(2^n \cdot n)$$

Las estructuras temporales empleadas durante la construcción y consulta de la tabla presentan complejidad $O(2n)O(2^n)O(2n)$.

Estructuras Dominantes

Las estructuras que más contribuyen al consumo de memoria son:

Los N-Cubos, utilizados para representar distribuciones de probabilidad multidimensionales.

La tabla de costos empleada por KGeoMIP.

Las estructuras de memoización utilizadas por KQNodes.

Las distribuciones marginales generadas durante la evaluación de particiones.

Estrategia	Complejidad Espacial
BruteForce	$(O(2^n \cdot n))$
KQNodes	$(O(n^2))$
KGeoMIP	$(O(2^n \cdot n))$

Tabla 4. Comparación de la complejidad espacial de cada estrategia

En conclusión, KQNodes presenta el menor consumo de memoria gracias al uso de estructuras de caché compactas, mientras que BruteForce y KGeoMIP requieren almacenar distribuciones o tablas cuyo tamaño crece exponencialmente con el número de variables del sistema. Esto convierte a KQNodes en la alternativa más eficiente desde el punto de vista espacial para sistemas de mayor tamaño.

Análisis de casos

Mejor Caso

El mejor caso ocurre cuando existe una partición que separa el sistema en grupos prácticamente independientes, produciendo una pérdida de información muy baja o incluso $\phi=0$.

En esta situación:

BruteForce puede finalizar rápidamente si encuentra la partición óptima en las primeras evaluaciones.

KQNodes y KGeoMIP identifican divisiones favorables desde las primeras iteraciones, reduciendo el número de evaluaciones necesarias.

Este comportamiento suele observarse en sistemas con estructura modular o con baja interdependencia entre sus componentes.

Peor Caso

El peor caso ocurre cuando el sistema presenta un alto grado de integración y todas las particiones generan pérdidas de información similares.

En este escenario:

- BruteForce debe evaluar prácticamente todas las particiones posibles antes de determinar la mejor solución.
- KQNodes y KGeoMIP continúan ejecutando todas las iteraciones necesarias para construir la k-partición, sin beneficiarse significativamente de sus heurísticas.

Este comportamiento es más frecuente en sistemas altamente conectados o con fuertes dependencias entre variables.

Influencia del Valor de k:

El valor de k también influye directamente en el rendimiento de los algoritmos.

- Valores pequeños de k requieren pocas divisiones y generan un menor número de evaluaciones.
- A medida que k aumenta, se incrementa el número de particiones candidatas que deben analizarse, aumentando el tiempo de ejecución y el consumo de memoria.

Por esta razón, los casos más costosos suelen presentarse cuando se combinan sistemas altamente integrados con valores elevados de k.

Comparación con Alternativas

La extensión a k-particiones fue diseñada para reducir el costo computacional asociado a la búsqueda exhaustiva, manteniendo una calidad de solución aceptable.

Método	Complejidad Temporal	Garantía de Óptimo
BruteForce	$(\Theta(2^{2n}))$	Sí
KQNodes (bipartición)	$(O(n^3))$	No

Método	Complejidad Temporal	Garantía de Óptimo
GeoMIP (bipartición)	$(O(2^n \cdot n^2))$	No
KQNodes	$(O(k \cdot n^3))$	No
KGeoMIP	$(O(k \cdot 2^n \cdot n^2))$	No

Tabla 5. Comparación de complejidad entre las estrategias implementadas y las alternativas existentes.

La estrategia BruteForce garantiza encontrar la solución óptima, pero su crecimiento exponencial limita su aplicación a sistemas pequeños. En contraste, KQNodes y KGeoMIP utilizan heurísticas para reducir significativamente el tiempo de ejecución, permitiendo analizar sistemas de mayor tamaño.

2.6 Detalles de Implementación

Métodos principales

A continuación se describen los métodos más relevantes para la ejecución de las estrategias implementadas.

`sia_preparar_subsistema(...)`

Prepara el subsistema sobre el cual se realizará la búsqueda de particiones.

Parámetros:

`estado`: estado inicial del sistema.

`condicion`: nodos condicionados.

`alcance`: conjunto de nodos de salida.

`mecanismo`: conjunto de nodos de entrada.

Retorno:

Actualiza internamente el subsistema y sus distribuciones marginales.

`distribucion_marginal()`

Calcula la distribución marginal asociada a un subsistema.

Retorno:

Arreglo NumPy con la distribución de probabilidad resultante.

`marginalizar(ejes)`

Reduce un N-Cubo eliminando las dimensiones especificadas.

Parámetros:

`ejes`: dimensiones a marginalizar.

Retorno:

Nuevo objeto `NCube` con dimensionalidad reducida.

`emd_efecto(u, v)`

Calcula la pérdida de información entre dos distribuciones de probabilidad.

Parámetros:

`u`: distribución original.

`v`: distribución particionada.

Retorno:

Valor numérico correspondiente a la distancia EMD.

`aplicar_estrategia(...)`

Método principal implementado por cada estrategia (KGeoMIP, KQNodes y BruteForce).

Funcionalidad:

Ejecuta el proceso de búsqueda de particiones.

Evalúa los candidatos generados.

Retorna la mejor solución encontrada.

Retorno:

Objeto `Solution`.

Dependencias externas

Las principales bibliotecas utilizadas por el proyecto se muestran en la siguiente tabla

Biblioteca	Uso
NumPy	Manipulación de matrices y distribuciones de probabilidad.
Colorama	Formateo y resaltado de mensajes en consola.
pyttsx3	Síntesis de voz para notificación de resultados.
PyInstrument	Análisis de rendimiento y profiling.

Tabla 6. Dependencias externas utilizadas por el sistema.

Aspectos de ingeniería de software

Logging

El sistema implementa mecanismos de registro mediante la clase `SafeLogger`, permitiendo almacenar información de ejecución y facilitar el diagnóstico de errores.

Profiling

Se utilizan herramientas de profiling para medir tiempos de ejecución y generar reportes de rendimiento durante los experimentos.

Validación de Entradas

Antes de iniciar el análisis se verifica la disponibilidad y consistencia de los archivos TPM utilizados como entrada.

Configuración Centralizada

Los parámetros globales del sistema se gestionan desde los módulos de configuración, evitando modificaciones directas en los algoritmos.

Tests implementados:

Para $n \leq 5$, se comparan los resultados de KQNodes y KGeoMIP con BruteForce.

Se verifica que $\phi(k=2)$ en las heurísticas sea igual o mayor que $\phi(k=2)$ de BruteForce (ninguna heurística puede ser mejor que el óptimo).

Se verifica la consistencia $k=2$: la 2-partición de KGeoMIP/KQNodes debe producir el mismo resultado que la bipartición original GeoMIP/QNodes.

#	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Iteración	Red	N	Variante	k	Partición KQNodes	Pérdida KQNodes	Tiempo KQNodes (s)	Partición Geométrica	Pérdida Geométrica	Tiempo Geométrico (s)	Δ Pérdida	Particiones coinciden
2	1	N2A	2	A	2	$\{+1, \emptyset, t\} \otimes \{B, t+1, a\}$	0,5	0,0930915	$\{t+1, b, t\} \otimes \{B, t+1, a\}$	0	0,0043025	-0,5	FALSO
3	2	N2A	2	A	3	$\{ \} \otimes \{ \emptyset, t+1, a, t \} \otimes \{ \}$	1	0,0096573	$\{ \} \otimes \{ \emptyset, t+1, b, t \} \otimes \{ \}$	0,5	0,0042916	-0,5	FALSO
4	3	N2A	2	A	4	$\{t+1, a, t\} \otimes \{B, t+1, c\}$	1	0,0129527	$\{t+1, \emptyset, t\} \otimes \{ \emptyset, t+1, a\}$	1	0,0047578	0	FALSO
5	4	N2A	2	A	5	$\{t+1, a, t\} \otimes \{B, t+1, c\}$	1	0,0126394	$\{t+1, \emptyset, t\} \otimes \{ \emptyset, t+1, a\}$	1	0,0044211	0	FALSO
6	5	N3A	3	A	2	$\{t+1, c, t\} \otimes \{C, t+1, a\}$	0,5	0,0203042	$\{+1, a, b, c, t\} \otimes \{B, t+1, a\}$	0,25	0,004317	-0,25	FALSO
7	6	N3A	3	A	3	$\{t\} \otimes \{B, t+1, \emptyset, t\} \otimes \{ \}$	0,75	0,02693	$\{ \} \otimes \{A, t+1, a, b, c, t\} \otimes \{ \}$	0,75	0,0062631	0	FALSO
8	7	N3A	3	A	4	$\{t+1, \emptyset, t\} \otimes \{ \emptyset, t+1, a\}$	1	0,0309318	$\{+1, \emptyset, t\} \otimes \{A, t+1, b\}$	0,75	0,0073179	-0,25	FALSO
9	8	N3A	3	A	5	$\{ \} \otimes \{A, t+1, \emptyset, t\} \otimes \{ \}$	1,5	0,0370581	$\{ \} \otimes \{ \emptyset, t+1, a, t \} \otimes \{ \}$	1,25	0,0076149	-0,25	FALSO
10	9	N3B	3	B	2	$\{t, b, t\} \otimes \{A, B, C, t+1, a\}$	0	0,0054889	$\{t+1, a, c, t\} \otimes \{ \emptyset, t+1, a\}$	0	0,0044436	0	FALSO
11	10	N3B	3	B	3	$\{ \} \otimes \{A, t+1, a, t\} \otimes \{ \}$	0,25	0,0181952	$\{ \} \otimes \{A, C, t+1, a, c, t\} \otimes \{ \}$	0,25	0,0077408	0	FALSO
12	11	N3B	3	B	4	$\{t+1, c, t\} \otimes \{ \emptyset, t+1, a\}$	0,53125	0,0246417	$\{t+1, \emptyset, t\} \otimes \{C, t+1, c\}$	0,5	0,0068428	-0,03125	FALSO
13	12	N3B	3	B	5	$\{ \} \otimes \{A, t+1, \emptyset, t\} \otimes \{ \}$	1,03125	0,0273426	$\{ \} \otimes \{C, t+1, c, t\} \otimes \{ \}$	0,78125	0,0105654	-0,25	FALSO
14	13	N3C	3	C	2	$\{t, \emptyset, t\} \otimes \{B, C, t+1, a\}$	0,5	0,0220805	$\{t+1, b, c, t\} \otimes \{C, t+1, a\}$	0	0,0045392	-0,5	FALSO
15	14	N3C	3	C	3	$\{ \} \otimes \{ \emptyset, t+1, c, t \} \otimes \{B, t+1, a\}$	0,5	0,0277094	$\{ \} \otimes \{A, t+1, c, t\} \otimes \{ \}$	0	0,0067657	-0,5	FALSO
16	15	N3C	3	C	4	$\{t+1, c, t\} \otimes \{ \emptyset, t+1, a\}$	1	0,0311493	$\{t+1, b, t\} \otimes \{ \emptyset, t+1, a\}$	0,5	0,0113472	-0,5	FALSO
17	16	N3C	3	C	5	$\{ \} \otimes \{ \emptyset, t+1, a, t \} \otimes \{ \}$	1	0,0349506	$\{ \} \otimes \{C, t+1, \emptyset, t\} \otimes \{ \}$	1	0,0113453	0	FALSO
18	17	N4A	4	A	2	$\{ \emptyset, t \} \otimes \{A, C, D, t+1, a\}$	0,25	0,052701	$\{+1, a, b, c, d, t\} \otimes \{B, t+1, a\}$	0,25	0,0074934	0	FALSO
19	18	N4A	4	A	3	$\{ \{C, t+1, \emptyset, t\} \otimes \{A, D, t+1, a\} \}$	0,75	0,0742066	$\{ \{A, D, t+1, a, b, c, d, t\} \}$	0,75	0,0110066	0	FALSO
20	19	N4A	4	A	4	$\{t, \emptyset, t\} \otimes \{ \emptyset, t+1, a, t \}$	0,75	0,0965626	$\{t, \emptyset, t\} \otimes \{A, D, t+1, b\}$	0,75	0,0134343	0	FALSO
21	20	N4A	4	A	5	$\{ \} \otimes \{ \emptyset, t+1, a, t \} \otimes \{ \}$	1,5	0,1114777	$\{ \} \otimes \{ \emptyset, t+1, a, t \} \otimes \{ \}$	1,25	0,0165052	-0,25	FALSO
22	21	N4B	4	B	2	$\{c, t\} \otimes \{A, B, C, D, t+1, a\}$	0	0,0185206	$\{t+1, b, c, d, t\} \otimes \{ \emptyset, t+1, a\}$	0	0,0071772	0	FALSO
23	22	N4B	4	B	3	$\{ \emptyset, t+1, a, t \} \otimes \{A, B, t+1, a\}$	0	0,0247556	$\{ \} \otimes \{A, B, t+1, c, d, t\} \otimes \{ \}$	0	0,0121709	0	FALSO
24	23	N4B	4	B	4	$\{a, t\} \otimes \{ \emptyset, t+1, d, t \}$	0,350000024	0,0352305	$\{t+1, b, t\} \otimes \{A, B, t+1, a\}$	0	0,0206653	-0,35	FALSO
25	24	N4B	4	B	5	$\{ \} \otimes \{ \emptyset, t+1, d, t \} \otimes \{A, t+1, a\}$	0,350000024	0,0402434	$\{t\} \otimes \{ \emptyset, t+1, c, t \} \otimes \{ \}$	0,175000012	0,0242892	-0,175	FALSO

Tabla 7. Resultados experimentales obtenidos para diferentes tamaños de red y valores de k .

2.7 Resultados Experimentales

Datasets utilizados

Las pruebas se realizaron utilizando matrices de probabilidad de transición (TPM) almacenadas en el directorio data/samples. Los experimentos consideraron redes de

diferentes tamaños, variando el número de nodos y el número de particiones k , con el fin de evaluar el comportamiento de las estrategias implementadas en distintos escenarios.

Métricas de Evaluación

Para evaluar el desempeño de los algoritmos se utilizaron las siguientes métricas:

Pérdida de información (ϕ): medida mediante la distancia EMD entre la distribución original y la distribución obtenida tras la partición.

Tiempo de ejecución: tiempo requerido para encontrar una solución.

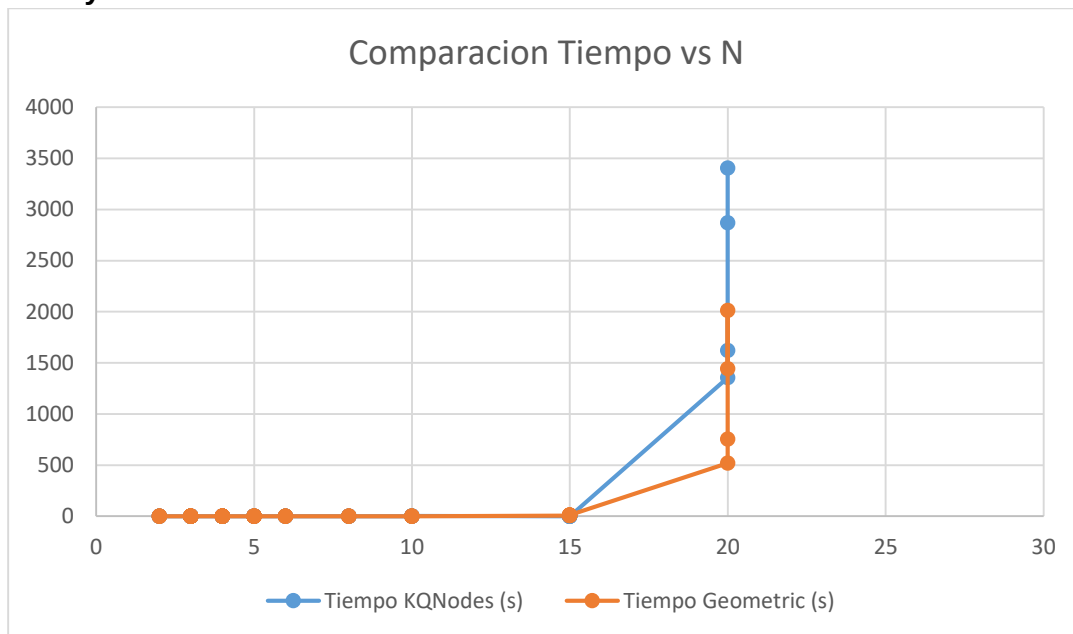
Error relativo: diferencia entre la solución heurística y la solución óptima obtenida mediante búsqueda exhaustiva, cuando esta última fue computacionalmente viable.

Tablas de Resultados

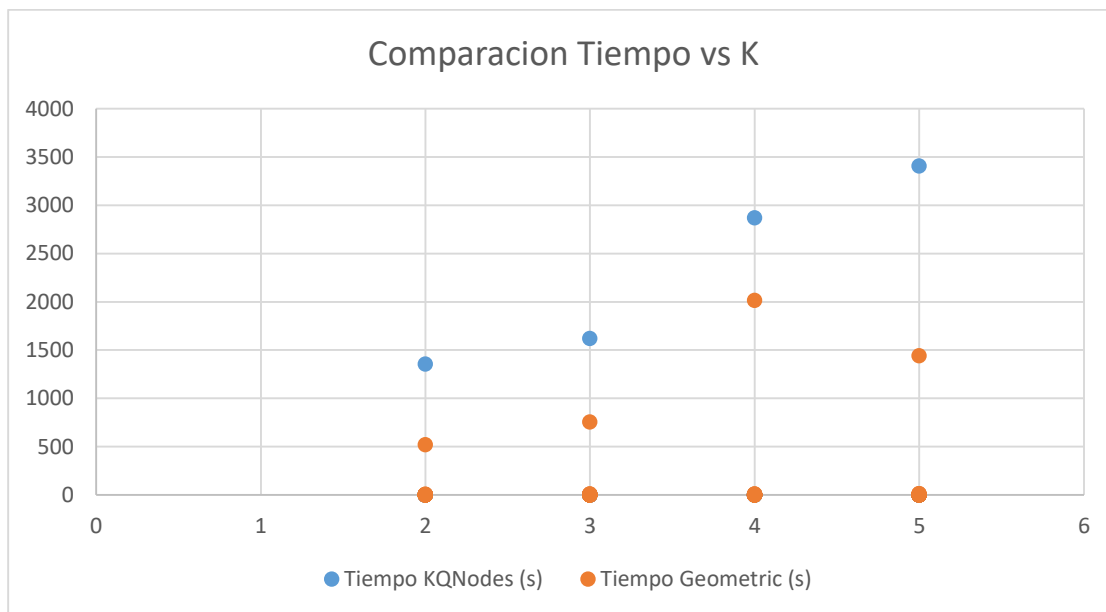
Los resultados obtenidos para las diferentes configuraciones de red y valores de k se presentan en las tablas anexas. En ellas se incluyen los valores de pérdida de información, las particiones encontradas y los tiempos de ejecución para cada estrategia.

	A	B	C	D	E	F	G	H	I	J	K	L	M	
1	Iteración	Red	N	Variante	k	Partición KQNodes	Pérdida KQNodes	Tiempo KQNodes	Partición Geométrica	Pérdida Geométrica	Tiempo Geométrica	ϕ	Pérdida	Particiones coincidentes
2	1	N2A	2	A	2	$\{1, 2\} \otimes \{B, t+1, 3\}$	0,5	0,0930915	$\{1, 2\} \otimes \{B, t+1, 3\}$	0	0,0043025	-0,5	FALSO	
3	2	N2A	2	A	3	$\{1, 2, 3\} \otimes \{A, t+1, 4\}$	1	0,0036573	$\{1, 2, 3\} \otimes \{A, t+1, 4\}$	0,5	0,0042916	-0,5	FALSO	
4	3	N2A	2	A	4	$\{1, 2, 3, 4\} \otimes \{B, t+1, 5\}$	1	0,0129527	$\{1, 2, 3, 4\} \otimes \{B, t+1, 5\}$	1	0,0047578	0	FALSO	
5	4	N2A	2	A	5	$\{1, 2, 3, 4, 5\} \otimes \{B, t+1, 6\}$	1	0,0126394	$\{1, 2, 3, 4, 5\} \otimes \{B, t+1, 6\}$	1	0,0044211	0	FALSO	
6	5	N3A	3	A	2	$\{1, 2, 3\} \otimes \{C, t+1, 4\}$	0,5	0,0203042	$\{1, 2, 3\} \otimes \{C, t+1, 4\}$	0,25	0,004317	-0,25	FALSO	
7	6	N3A	3	A	3	$\{1, 2, 3, 4\} \otimes \{B, t+1, 5\}$	0,75	0,02693	$\{1, 2, 3, 4\} \otimes \{B, t+1, 5\}$	0,75	0,0062631	0	FALSO	
8	7	N3A	3	A	4	$\{1, 2, 3, 4, 5\} \otimes \{B, t+1, 6\}$	1	0,0303318	$\{1, 2, 3, 4, 5\} \otimes \{B, t+1, 6\}$	0,75	0,0073173	-0,25	FALSO	
9	8	N3A	3	A	5	$\{1, 2, 3, 4, 5, 6\} \otimes \{B, t+1, 7\}$	1,5	0,0370581	$\{1, 2, 3, 4, 5, 6\} \otimes \{B, t+1, 7\}$	1,25	0,0076149	-0,25	FALSO	
10	9	N3B	3	B	2	$\{1, 2, 3\} \otimes \{A, B, C, t+1, 4\}$	0	0,0054889	$\{1, 2, 3\} \otimes \{A, B, C, t+1, 4\}$	0	0,0044436	0	FALSO	
11	10	N3B	3	B	3	$\{1, 2, 3, 4\} \otimes \{A, t+1, 5\}$	0,25	0,0181952	$\{1, 2, 3, 4\} \otimes \{A, t+1, 5\}$	0,25	0,0077408	0	FALSO	
12	11	N3B	3	B	4	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	0,53125	0,0246417	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	0,5	0,0068428	-0,0313	FALSO	
13	12	N3B	3	B	5	$\{1, 2, 3, 4, 5, 6\} \otimes \{A, t+1, 7\}$	1,03125	0,0273426	$\{1, 2, 3, 4, 5, 6\} \otimes \{A, t+1, 7\}$	0,78125	0,0105654	-0,25	FALSO	
14	13	N3C	3	C	2	$\{1, 2, 3\} \otimes \{B, C, t+1, 4\}$	0,5	0,0220805	$\{1, 2, 3\} \otimes \{B, C, t+1, 4\}$	0	0,0045392	-0,5	FALSO	
15	14	N3C	3	C	3	$\{1, 2, 3, 4\} \otimes \{A, t+1, 5\}$	0,5	0,0277094	$\{1, 2, 3, 4\} \otimes \{A, t+1, 5\}$	0	0,0067657	-0,5	FALSO	
16	15	N3C	3	C	4	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	1	0,0311493	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	0,5	0,0113472	-0,5	FALSO	
17	16	N3C	3	C	5	$\{1, 2, 3, 4, 5, 6\} \otimes \{A, t+1, 7\}$	1	0,0349506	$\{1, 2, 3, 4, 5, 6\} \otimes \{A, t+1, 7\}$	1	0,0113453	0	FALSO	
18	17	N4A	4	A	2	$\{1, 2\} \otimes \{A, C, D, t+1, 3\}$	0,25	0,052701	$\{1, 2\} \otimes \{A, b, c, d, t+1, 3\}$	0,25	0,0074934	0	FALSO	
19	18	N4A	4	A	3	$\{1, 2, 3\} \otimes \{A, t+1, 4\}$	0,75	0,0742066	$\{A, D, t+1, a, b, c, d, t+1, 4\}$	0,75	0,0110066	0	FALSO	
20	19	N4A	4	A	4	$\{1, 2, 3, 4\} \otimes \{A, t+1, 5\}$	0,75	0,0965626	$\{1, 2\} \otimes \{A, D, t+1, 5\}$	0,75	0,0134343	0	FALSO	
21	20	N4A	4	A	5	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	1,5	0,1141777	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	1,25	0,0165052	-0,25	FALSO	
22	21	N4B	4	B	2	$\{1, 2\} \otimes \{A, B, C, D, t+1, 3\}$	0	0,0185206	$\{1, 2\} \otimes \{b, c, d, t+1, 3\}$	0	0,0071772	0	FALSO	
23	22	N4B	4	B	3	$\{1, 2, 3\} \otimes \{A, B, t+1, 4\}$	0	0,0247556	$\{1, A, B, t+1, c, d, t+1, 4\}$	0	0,0121709	0	FALSO	
24	23	N4B	4	B	4	$\{1, 2, 3, 4\} \otimes \{A, t+1, 5\}$	0,350000024	0,0352305	$\{1, 2, 3, 4\} \otimes \{A, B, t+1, 5\}$	0	0,0206653	-0,35	FALSO	
25	24	N4B	4	B	5	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	0,350000024	0,0402434	$\{1, 2\} \otimes \{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	0,175000012	0,0242892	-0,175	FALSO	
26	25	N4C	4	C	2	$\{1, 2\} \otimes \{A, B, D, t+1, 3\}$	0,125	0,0461964	$\{1, 2, 3, 4, 5\} \otimes \{C, t+1, 3\}$	0,125	0,0068804	0	FALSO	
27	26	N4C	4	C	3	$\{1, 2, 3\} \otimes \{D, t+1, 4\}$	0,625	0,0938528	$\{A, D, t+1, a, b, c, d, t+1, 4\}$	0,625	0,0106455	0	FALSO	
28	27	N4C	4	C	4	$\{1, 2, 3, 4\} \otimes \{B, t+1, 5\}$	1,375	0,0861561	$\{1, 2, 3\} \otimes \{A, t+1, a, b, t+1, 5\}$	1,125	0,0147363	-0,25	FALSO	
29	28	N4C	4	C	5	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	1,375	0,0886424	$\{1, 2\} \otimes \{D, t+1, 6\}$	1,125	0,0164839	-0,25	FALSO	
30	29	N5A	5	A	2	$\{1, 2\} \otimes \{A, B, C, D, E, t+1, 3\}$	0	0,0485518	$\{1, 2\} \otimes \{a, b, c, d, e, t+1, 3\}$	0	0,0111612	0	FALSO	
31	30	N5A	5	A	3	$\{1, 2, 3\} \otimes \{A, B, t+1, 4\}$	0	0,1308052	$\{A, B, C, E, t+1, a, b, c, d, t+1, 4\}$	0	0,0142243	0	FALSO	
32	31	N5A	5	A	4	$\{1, 2, 3, 4\} \otimes \{B, t+1, 5\}$	0,25	0,1512831	$\{1, 2\} \otimes \{A, C, F, t+1, 5\}$	0,25	0,0223465	0	FALSO	
33	32	N5A	5	A	5	$\{1, 2, 3, 4, 5\} \otimes \{C, t+1, 6\}$	0,75	0,1757802	$\{B, t+1, a, 1, 2\} \otimes \{A, t+1, 6\}$	0,75	0,0273523	0	FALSO	
34	33	N5B	5	B	2	$\{1, 2\} \otimes \{B, C, D, t+1, 3\}$	0	0,1051243	$\{1, 2\} \otimes \{b, c, d, t+1, 3\}$	0	0,0108123	0	VERDADERO	
35	34	N5B	5	B	3	$\{1, 2, 3\} \otimes \{A, B, t+1, 4\}$	0,125	0,1539451	$\{A, B, D, t+1, b, c, d, t+1, 4\}$	0,125	0,018193	0	FALSO	
36	35	N5B	5	B	4	$\{1, 2, 3, 4\} \otimes \{B, t+1, 5\}$	0,375	0,1918063	$\{E, t+1, 1, 2\} \otimes \{A, t+1, 5\}$	0,375	0,0235052	0	FALSO	
37	36	N5B	5	B	5	$\{1, 2, 3, 4, 5\} \otimes \{A, t+1, 6\}$	0,625	0,204327	$\{C, t+1, 1, 2\} \otimes \{B, t+1, 6\}$	0,625	0,02325	0	FALSO	
38	37	N6A	6	A	2	$\{1, 2\} \otimes \{A, B, C, D, F, t+1, 3\}$	0,46875	0,2175112	$\{1, 2, 3, 4, 5, 6\} \otimes \{A, t+1, 3\}$	0,4375	0,0174973	-0,0313	FALSO	
39	38	N6A	6	A	3	$\{1, 2, 3\} \otimes \{A, B, D, t+1, 4\}$	0,353125	0,2864683	$\{B, C, F, t+1, a, b, c, d, t+1, 4\}$	0,30625	0,0261488	-0,0469	FALSO	

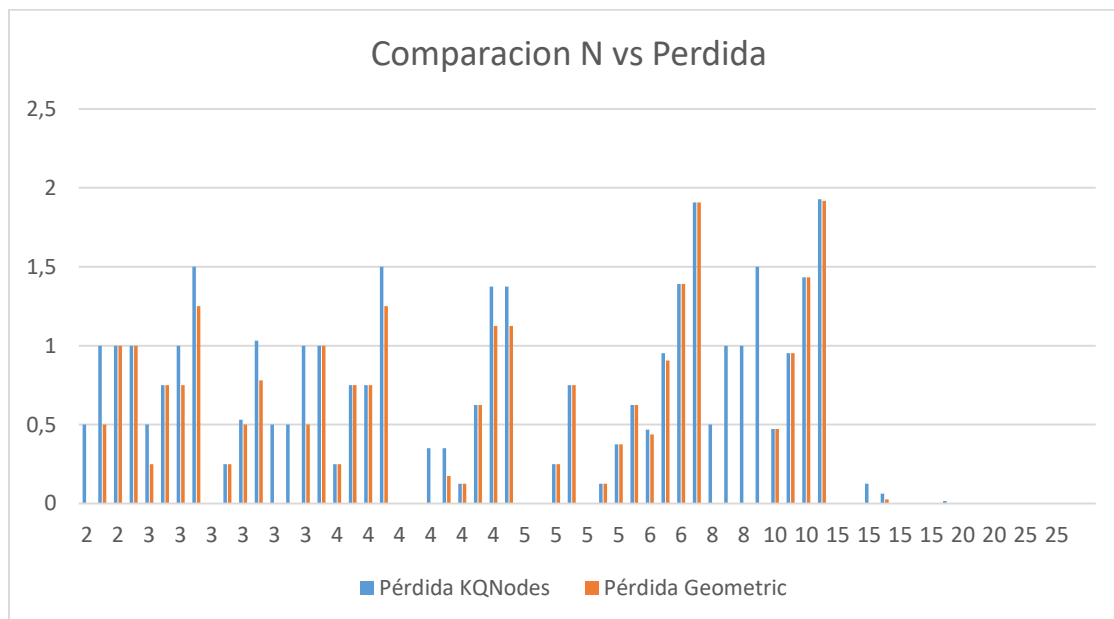
Tabla 8. Resultados experimentales obtenidos para KQNodes y KGeoMIP en diferentes tamaños de red y valores de k .

Gráficas y visualizaciones

Grafica 12. Comparación Tiempo vs N con las dos estrategias



Grafica 13. Comparación Tiempo vs K con las dos estrategias



Grafica 14. Comparación N vs Perdida con las dos estrategias

Análisis de Resultados

Los resultados muestran que ambas estrategias son capaces de generar k-particiones válidas manteniendo tiempos de ejecución considerablemente menores que una búsqueda exhaustiva.

En general, KQNodes presenta un mejor comportamiento en términos de escalabilidad, mientras que KGeoMIP ofrece resultados competitivos aprovechando la estructura geométrica del espacio de estados. A medida que aumenta el número de nodos y el valor de k, el costo computacional de ambas estrategias también aumenta debido al crecimiento del espacio de búsqueda.

Validación de Correctitud

La correctitud de las implementaciones se verificó comparando los resultados obtenidos con los generados por BruteForce en los casos donde la búsqueda exhaustiva fue factible.

Además, se comprobó la consistencia para $k=2$, verificando que las extensiones desarrolladas producen resultados equivalentes a los algoritmos originales de bipartición.

2.8 Limitaciones y Trabajo Futuro

Limitaciones Conocidas

Las heurísticas implementadas no garantizan encontrar siempre la partición óptima, especialmente en sistemas de gran tamaño o con estructuras altamente interconectadas. Además, el enfoque geométrico presenta limitaciones de escalabilidad debido al

crecimiento exponencial de memoria requerido para procesar sistemas con un número elevado de nodos.

Supuestos y Restricciones

La solución asume que los datos de entrada cumplen con el formato esperado y que las relaciones entre los nodos pueden modelarse mediante las matrices de transición utilizadas. Asimismo, los resultados obtenidos dependen del estado inicial seleccionado para el análisis, por lo que diferentes configuraciones pueden producir particiones distintas.

Mejoras Potenciales

Como trabajo futuro, se podría incorporar procesamiento paralelo para reducir los tiempos de ejecución, implementar métricas causales más completas y desarrollar mecanismos de optimización que permitan explorar de manera más eficiente el espacio de búsqueda. También sería útil mejorar la validación de diferentes modos de operación del sistema.

Direcciones de Investigación Futura

Se propone estudiar métodos que ofrezcan garantías teóricas sobre la calidad de las particiones obtenidas, extender el enfoque a sistemas con múltiples estados por nodo y analizar el comportamiento de las heurísticas en escenarios de mayor complejidad. Además, sería interesante realizar comparaciones con otras herramientas y enfoques utilizados en la Teoría de la Información Integrada (IIT).

2.9 Apéndices Técnicos

Demostraciones

Equivalencia utilizada en el cálculo de EMD

Durante la implementación se utilizó la propiedad de que, para distribuciones binarias independientes, la distancia EMD puede calcularse a partir de las diferencias entre las probabilidades marginales de cada nodo. Esta simplificación reduce significativamente el costo computacional sin afectar la interpretación de los resultados en los casos evaluados.

La equivalencia se basa en que la distribución conjunta puede expresarse como el producto de las distribuciones individuales de los nodos. Bajo esta condición, la distancia total entre dos distribuciones puede descomponerse como la suma de las distancias de cada variable individual, permitiendo un cálculo más eficiente de la información integrada.

Esta propiedad fue utilizada como fundamento para optimizar el cálculo de las métricas empleadas por los algoritmos desarrollados.

No se realizaron demostraciones formales adicionales, ya que el proyecto se enfocó en la implementación y evaluación computacional de los algoritmos propuestos.

Detalles Algorítmicos Adicionales Generación de Biparticiones

La generación de biparticiones consiste en crear todas las combinaciones posibles de división entre los nodos del mecanismo y del alcance, descartando aquellas particiones triviales donde uno de los conjuntos queda vacío.

Pseudocódigo:

GENERAR_BIPARTICIONES()

Generar todas las combinaciones posibles de nodos.

Dividir los nodos en dos subconjuntos.

Verificar que ambos subconjuntos sean no vacíos.

Almacenar la partición válida.

Retornar el conjunto de particiones.

```

FUNCIÓN GENERAR_BIPARTICIONES(m, n)
    // Genera todas las  $2^{(m-1)} \times 2^n$  biparticiones no triviales
    PARA combo_alcance EN product({0,1}, repetición=m):
        PARA combo_mec EN product({0,1}, repetición=n):
            SI NO es trivial(combo_alcance, combo_mec):
                RETORNAR (alcance_A, alcance_B, mec_A, mec_B)
    FIN
  
```

Construcción de Caminos de Hamming

El algoritmo genera secuencias de estados que permiten transformar un estado inicial en un estado final modificando un único bit en cada paso.

```

FUNCIÓN CONSTRUIR_CAMINOS_HAMMING( $s_0$ ,  $s_f$ , n)
    caminos  $\leftarrow \{0: [s_0]\}$ 
    visitados  $\leftarrow \{s_0\}$ 
  
```

```

PARA nivel DESDE 1 HASTA n:
  PARA estado EN caminos[nivel-1]:
    PARA bit i DESDE 0 HASTA n-1:
      si estado[i] != s_f[i]:
        nuevo ← estado con bit i invertido
        SI nuevo NO EN visitados:
          caminos[nivel].agregar(nuevo)
          visitados.agregar(nuevo)

RETORNAR caminos

```

Resultados Experimentales

Durante las pruebas se evaluó el comportamiento de las heurísticas implementadas en diferentes configuraciones del sistema.

Los resultados permitieron verificar:

- Correcta generación de particiones candidatas.
- Reducción del tiempo de ejecución respecto al método exhaustivo.
- Consistencia en los valores de información integrada obtenidos.
- Comportamiento estable para distintos tamaños de sistemas evaluados.

Referencias Bibliográficas

Tononi, G. (2004). *An Information Integration Theory of Consciousness*. BMC Neuroscience, 5(1), 42.

Oizumi, M., Albantakis, L., & Tononi, G. (2014). *From the phenomenology to the mechanisms of consciousness: Integrated Information Theory 3.0*. PLOS Computational Biology, 10(5).

Mayner, W. G., et al. (2018). *PyPhi: A toolbox for integrated information theory*. PLOS Computational Biology, 14(7).

NumPy Development Team. *NumPy Documentation*. <https://numpy.org>

Python Software Foundation. *Python Documentation*. <https://docs.python.org>

3. Uso de Inteligencia Artificial Generativa

Durante el desarrollo del proyecto se utilizaron herramientas de inteligencia artificial generativa, principalmente ChatGPT y Claude, como apoyo en tareas de investigación, implementación, depuración y documentación.

Las consultas realizadas estuvieron orientadas a comprender conceptos relacionados con la Teoría de la Información Integrada (IIT), analizar algoritmos, resolver dudas de programación en Python y generar estructuras iniciales de pseudocódigo y documentación técnica

Ejemplos de consultas realizadas:

- "Analiza la complejidad temporal de este algoritmo."
- "Genera pseudocódigo para un algoritmo de generación de biparticiones."
- "Ayúdame a estructurar la documentación técnica de un proyecto de algoritmos."